Ministerul Educației și Cercetării al Republicii Moldova

Universitatea Tehnică a Moldovei

Facultatea Calculatoare, Informatică și Microelectronică

Laboratory Work 2

Building and Exploring the xv6 Operating System

Operating Systems

Elaborated by:
Student, group FAF-241
Cațț Chiril

Verified:
asist. univ. Reițman Patricia

Chișinău – 2026

Contents

1 Introduction **1**

1.1 Objective 1

1.2 Tasks 1

1.3 Environment and Tools 1

2 Building and Booting xv6 **2**

3 Using xv6 as a Unix System **3**

3.1 Built-in Programs 3

3.2 Pipelines and Interprocess Communication 4

3.3 Comparison with a Linux Shell 4

4 Reading the xv6 Source **5**

4.1 User and Kernel Code 5

4.2 System Calls Used by cat 5

4.3 Locating sys read 6

5 Implementing the sleep User Program **6**

5.1 Program Source 6

5.2 Adding the Program to the Build 7

5.3 Execution Test 7

6 Discussion **7**

7 Conclusion **8**

1 Introduction

xv6 is a small teaching operating system that reimplements the central ideas of Unix Version 6 for the RISC-V architecture. Unlike a production operating system, xv6 is intentionally compact enough for its kernel, user programs, system-call interface, filesystem, memory manager, and scheduler to be studied directly. It runs under the QEMU machine emulator and provides a minimal Unix-like shell.

This laboratory work moves from observing Linux externally to examining both sides of an operating-system interface. The user programs in `user/` request services, while the privileged code in `kernel/` implements those services. The practical work included compiling and booting xv6, using its standard programs, inspecting source code, and adding a new user-space program that invokes the existing `sleep` system call.

1.1 Objective

The objective is to build and boot xv6, become familiar with its Unix-like environment, identify the boundary between user and kernel code, trace system calls used by a program, and implement a working `sleep` command.

1.2 Tasks

The work was divided into four parts:

1. compile xv6 and boot it in QEMU;
2. run built-in user programs and test a shell pipeline;
3. inspect `user/cat.c`, the system-call declarations, and the kernel implementation of `sys_read`;
4. implement `user/sleep.c`, add it to `UPROGS`, rebuild xv6, and test the result.

1.3 Environment and Tools

The experiment used the course Linux environment with the RISC-V cross-compiler and QEMU. The xv6 source tree was organized around the two directories summarized in Table 1.

Table 1: Principal xv6 source directories

Directory	Role
`kernel/`	Privileged operating-system code, including process scheduling, memory management, filesystem code, device handling, and system-call handlers.
`user/`	Unprivileged programs and user-space support code, including the shell, `ls`, `cat`, and the new `sleep` program.

2 Building and Booting xv6

The source tree was compiled and started with:

```
cd ~/xv6-riscv
make qemu
```

The build creates the RISC-V kernel and xv6 user programs, then launches QEMU with the generated disk image. The successful boot sequence printed `xv6 kernel is booting`, started the available hardware threads (harts), launched the initial process, and displayed the xv6 shell prompt.

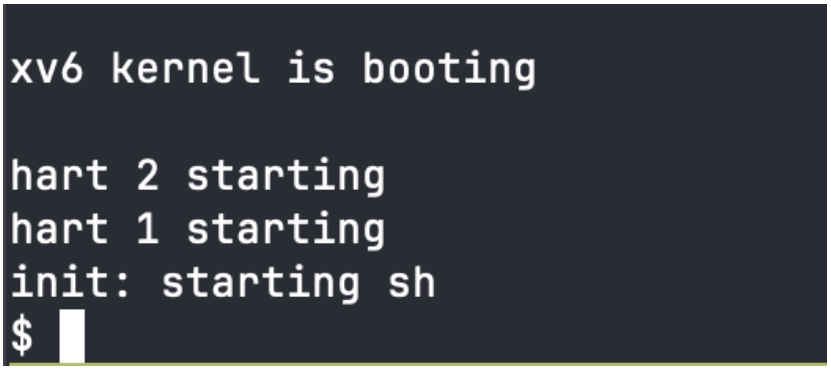


Figure 1: Successful xv6 boot followed by the initial shell prompt

The lines `hart 1 starting` and `hart 2 starting` show secondary RISC-V processor cores entering the kernel. The message `init: starting sh` confirms that the first user-level initialization process launched the shell. Reaching the `$` prompt verifies that the kernel, filesystem image, console device, init program, and shell all function together.

QEMU is exited with its monitor escape sequence: press Ctrl-A, release it, and then press X. This sequence is handled by QEMU rather than by the xv6 shell.

3 Using xv6 as a Unix System

3.1 Built-in Programs

The `cat README` command successfully opened the file, repeatedly read its contents, wrote them to standard output, and returned control to the shell.

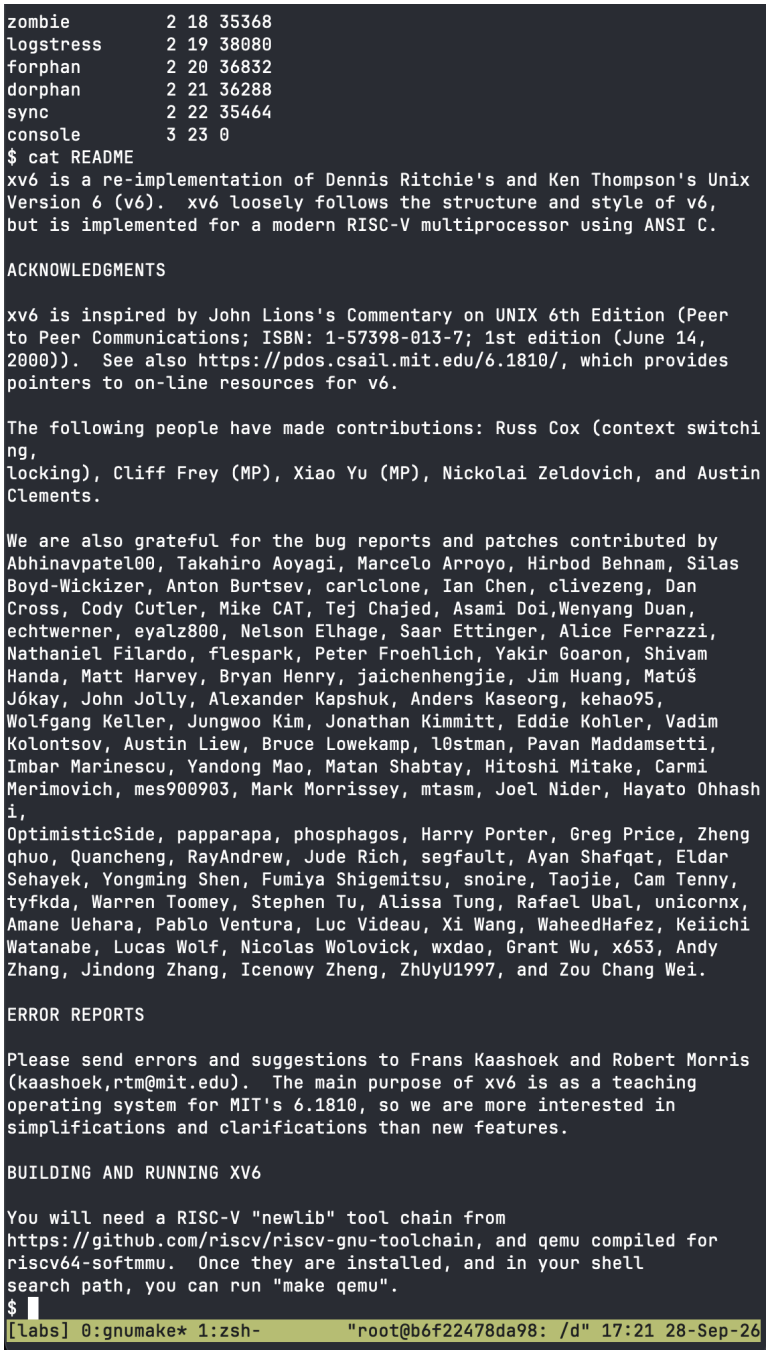


Figure 2: Reading the xv6 README file with the cat user program

The xv6 filesystem contains familiar Unix programs. The directory listing used during the laboratory included programs such as `grep`, `sh`, `rm`, and `cat`; any three of these satisfy

the observation task. Their interfaces are intentionally similar to their Linux counterparts, although the implementations and supported options are much smaller.

The README identifies xv6 as a modern RISC-V reimplementation of Unix Version 6 and describes the toolchain and QEMU requirement. More importantly for this experiment, the successful output demonstrates cooperation between the shell, the `cat` process, system calls, the xv6 filesystem, and the console driver.

3.2 Pipelines and Interprocess Communication

The pipeline `ls | grep sh` connects the standard output of `ls` to the standard input of `grep`. The result contains the directory entry for the xv6 shell.

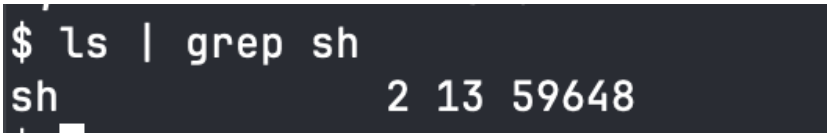


Figure 3: Using an xv6 pipeline to select the sh directory entry

Two operating-system capabilities are essential for this pipeline:

1. **Process management:** the shell must create and coordinate separate `ls` and `grep` processes, normally using `fork`, `exec`, and `wait`.
2. **Kernel-supported interprocess communication:** the kernel must create a pipe with readable and writable file descriptors and transfer bytes safely from one process to the other.

Standard input and output provide the conventional file descriptors that the shell redirects to the pipe endpoints. The pipe itself is the kernel object that buffers and transports the data.

3.3 Comparison with a Linux Shell

The xv6 shell follows the same core Unix model as a Linux shell: it launches programs, redirects file descriptors, constructs pipelines, and waits for child processes. It is considerably simpler, however, because it omits most interactive conveniences and language features of shells such as Bash, including command history, completion, job control, extensive expansion rules, scripting constructs, and configurable prompts.

4 Reading the xv6 Source

4.1 User and Kernel Code

Code in `user/` runs without kernel privileges and accesses protected resources only through system calls. Code in `kernel/` runs in supervisor mode, controls the hardware, manages address spaces and processes, and validates and serves system-call requests. On RISC-V, this distinction is more accurately described as user mode and supervisor mode rather than the x86-specific terms ring 3 and ring 0.

The boundary protects the whole system: a user program cannot directly manipulate page tables, devices, or another process's memory. Instead, it places arguments according to the xv6 calling convention and executes a trap into the kernel. The kernel validates the request, performs the operation, and returns a result.

4.2 System Calls Used by cat

The program `user/cat.c` uses the following system calls:

Table 2: System calls used by the xv6 cat program

Call	Request made to the kernel
`open`	Open a named file and return a file descriptor.
`read`	Copy bytes from a file descriptor into a user-space buffer.
`write`	Copy bytes from a user-space buffer to a file descriptor, including standard output.
`close`	Release an open file descriptor.
`exit`	Terminate the process and provide an exit status.

The program also calls `fprintf` to format error messages. `fprintf` is a user-space library routine, not a direct system call; it ultimately emits the formatted bytes through `write`. File descriptor 1 is standard output and file descriptor 2 is standard error. Reading continues until `read` returns zero for end of file or a negative value for an error.

A simplified outline of the important control flow is:

```
while ((n = read(fd, buf, sizeof(buf))) > 0) {
    if (write(1, buf, n) != n) {
        fprintf(2, "cat: write error\n");
        exit(1);
    }
}
```

4.3 Locating sys read

The declarations visible to user programs are listed in `user/user.h`. The kernel-side handler `sys_read` was located in `kernel/sysfile.c` at line 69 in the inspected source tree. Its responsibility is to retrieve and validate the system-call arguments before delegating the actual file read to the kernel's file layer.

This tracing exercise shows both sides of the interface: `user/cat.c` calls `read`, the system-call mechanism transfers control into the kernel, and `sys_read` handles the privileged operation.

5 Implementing the sleep User Program

5.1 Program Source

The new `user/sleep.c` program accepts exactly one argument, converts it from text to an integer number of clock ticks, invokes the existing `sleep` system call, and exits successfully after the delay.

Listing 1: Implementation of user/sleep.c

```
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
    if (argc != 2) {
        fprintf(2, "usage: sleep <ticks>\n");
        exit(1);
    }

    sleep(atoi(argv[1]));
    exit(0);
}
```

The argument check prevents access to a missing `argv[1]` and gives the user a clear usage message. `atoi` converts the decimal argument, and the system call blocks the process for the requested number of xv6 timer ticks. While the process sleeps, the scheduler can run other processes.

5.2 Adding the Program to the Build

xv6 user programs must be included in the `UPROGS` list in the Makefile so that they are compiled and copied into the filesystem image. The following entry was added with the same tab indentation and trailing backslash as the neighboring entries:

```
UPROGS=\
    $U/_cat\
    $U/_echo\
    $U/_sleep\
    ...
```

The underscore is part of the build target name used by the xv6 Makefile; inside xv6 the resulting command is invoked simply as `sleep`.

5.3 Execution Test

After rebuilding and booting xv6, the command `sleep 100` paused for the requested interval and then returned to the shell prompt without printing an error.



Figure 4: Running the implemented sleep command for 100 clock ticks

This behavior verifies that the program was compiled, included in the filesystem image, loaded as a user process, and able to invoke the kernel's `sleep` service. A complete negative test is `sleep` without an argument; it should print `usage: sleep <ticks>` to file descriptor 2 and terminate with status 1.

6 Discussion

The experiment connected several operating-system concepts that are often studied separately:

- **Boot and initialization:** QEMU provides emulated RISC-V hardware, the xv6 kernel initializes it, and `init` starts the first shell.
- **Processes and pipes:** the shell creates programs as separate processes and connects them with kernel-managed file descriptors.

- **Protection boundary:** user programs execute without privilege, while the kernel validates and fulfills system-call requests in supervisor mode.
- **Program integration:** adding `sleep.c` requires both correct source code and a Makefile entry so the executable becomes part of the xv6 filesystem image.

The `cat` and `sleep` examples reveal the same general pattern. Each user program performs a small amount of application logic and depends on system calls for protected operations. `cat` requests file and console I/O, while `sleep` requests timer-based blocking. The kernel provides the mechanism; the user program determines how it is used.

7 Conclusion

The laboratory work successfully built and booted xv6 under QEMU, exercised its shell and standard user programs, verified that pipelines provide communication between processes, examined the separation between `user/` and `kernel/`, and traced the system calls used by `cat`. The source inspection showed that `sys_read` is implemented in `kernel/sysfile.c` and clarified that `fprintf` is a library routine built on top of the system-call interface.

The new `sleep` program validated the complete path from user code to kernel service. It checked its arguments, converted the requested delay, invoked the existing system call, and ran successfully after being added to `UPROGS`. This provides a practical foundation for later work that modifies not only xv6 user programs but also the kernel itself.